

IS CAPSTONE – GROUP 10

PRESENTATION PLAN

Group 1 – Prisca Louis, Mariah Weldon, Felix Botero, Erik Gijlstra

Introduction:

Slide Title:

Group 10 – Integration with Payment Provider & E-Commerce Payment Flows

Slide Text (for the slide):

- **Erik Gijlstra** – Backend Server Developer & Warehouse Settlement UI
- **Mariah Weldon** – Payment Processing (Checkout) UI
- **Prisca Louis** – Order History UI
- **Felix Botero** – Database Design & Data Flow

Background:

Mariah:

Favorite Part of Development:

Most Challenging Part:

Erik:

Favorite Part of Development:

Most Challenging Part:

Prisca:

Favorite Part of Development:

Most Challenging Part:

Felix:

Favorite Part of Development:

Most Challenging Part:

Demonstration:

Payment Processing UI: Mariah

Goal: Demonstrate the customer-facing side of the payment flow — from form completion and validation to API authorization and session logging.

Demonstration Flow (about 2 minutes)

Step 1 – Introduce the Checkout Form

“Here’s our Checkout page.

It collects all the standard customer and payment information you’d see on an online store — name, address, city, ZIP, and payment details.”

(Scroll to show the split between Shipping Details and Payment Details)

- **Highlight the form split** (“We separated shipping and payment info for realism.”)
- **Point out the U.S. state selector** (“We included a state dropdown to simulate full address entry.”)

Step 2 – Billing Same as Shipping

“We also added a ‘*Billing same as Shipping*’ checkbox — this hides or shows the billing section, just like a real checkout.”

(Quickly toggle it once to show/hide the extra section)

Step 3 – Card Entry and Validation

“Now I’ll start entering a card number. As I type, the system automatically detects the card brand and formats the number.”

- Type “4...” → Visa badge/icon appears.
- Continue typing → auto-spaces appear in groups of four.

“You can see the card type updates in real time and the input is automatically grouped for readability.”

Step 4 – Expiry and CVV

“The expiry date automatically formats to *MM/YY*, and once it’s complete, the cursor jumps straight to the CVV field.”

(Demonstrate that quick flow; if possible, intentionally type an expired date to trigger error.)

“If I try to submit with an expired date or invalid CVV, it gives clear, field-level error messages.”

(Show red validation message for expiry or CVV — keep it brief.)

Step 5 – Submitting a Payment

“Now I’ll submit a valid payment.”

- Click “Pay Now.”

- Spinner overlay appears briefly.
- Success or error banner shows at the top.

“The app calls our backend API, which simulates a payment provider. It returns one of four outcomes — *success*, *insufficient funds*, *incorrect details*, or *server error*.

Here we got a successful authorization with a clear green confirmation banner.”

Step 6 – Session Log

“On the right-hand side, we have a live session log that tracks every authorization attempt for this session — including the order ID, amount, time, and outcome.”

(Scroll that panel briefly; point out the colored badge.)

Closing Transition Line

“So this page handles the full customer checkout process — complete with realistic validation, live formatting, and instant feedback — and then passes the authorization data to our backend for settlement.”

Summary of Features Demonstrated

Category	Feature	Demo Step
Required	Full Checkout Form	Step 1
Required	Client-Side Validation	Steps 3–4
Required	Formatted Inputs (Card/Expiry)	Step 3
Required	API Call / Mock Provider	Step 5
Required	Outcome Feedback Banner	Step 5
Required	Session Log Panel	Step 6
Bonus	Form Split (Shipping vs Payment)	Step 1
Bonus	U.S. State Selector	Step 1
Bonus	Billing Same as Shipping Checkbox	Step 2
Bonus	Card Type Icon + Live Update	Step 3

Bonus	Dynamic Error Messaging	Step 4
Bonus	Auto-Advance from Expiry → CVV	Step 4

Warehouse Settlement UI: Erik

Goal: Demonstrate the internal workflow after checkout — how warehouse staff validate, settle, and finalize authorized orders through the UI (backed by your Node/Express API + SQLite DB).

Demonstration Flow (~2 minutes)

Step 1 – Introduce the Warehouse Role

“After a payment is authorized, the warehouse team uses this screen to finalize the transaction by entering the settlement amount.”

(Briefly describe the top form layout — Order ID and Amount fields.)

Step 2 – Load an Order

“I’ll start by loading an existing authorized order.”

- Enter known Order ID (from Mariah’s test).
- Press **Load** → spinner appears → details populate below.
- Point out:
 - Order ID
 - Status badge “Authorized”
 - Authorized amount
 - “Available to Settle” value

Step 3 – Settle a Valid Amount

“Now I’ll enter a valid settlement amount that’s within the authorized total.”

- Type an amount less than or equal to remaining.
- Click **Settle** → shows spinner → success alert appears.
- Remaining amount updates dynamically.

“The API validated the amount, recorded the settlement in the database, and immediately returned the updated available balance.”

Step 4 – Show an Error Case

“If I try to over-settle — say I enter more than what’s available — it catches that automatically.”

- Enter an excessive amount → shows red error message:
“Amount exceeds available (\$___).”

“All of these validations are handled server-side but reported instantly to the UI.”

Step 5 – Settlement History

“Below that, we have a running list of settlements for this order — showing the time, amount, and outcome badges.”

(Scroll to show previous settlements.)

Step 6 – Optional Quick Reload

“If I reload this order again, you’ll see the same updated data.
Everything’s persisted in the SQLite database through our backend.”

(Press Load again — details reload instantly to prove persistence.)

Closing Transition

“So, this page lets warehouse staff safely settle payments with live validation against the authorized amount, all powered by our backend API.

Summary of Demonstrated Features

Category	Feature	Demo Step
Required	Load Order via API	Step 2
Required	Validate Authorized Status	Step 3
Required	Settlement Submission + Backend Validation	Step 3

Required	Error Codes & Alerts	Step 4
Required	Settlement History	Step 5
Bonus	Remaining Balance Preview	Step 3
Bonus	Status Badge Consistency	Step 2
Bonus	Input-Group “Load” Button	Step 2
Bonus	Shake Animation for Invalid Order	Step 4
Bonus	In-Memory Cache	Step 6
Bonus	Responsive Portal-Card Layout	Throughout

Order History UI: Prisca

Goal: Demonstrate how users can view, search, and filter all orders after checkout and settlement—proving the full workflow from customer payment to back-office visibility.

Demonstration Flow (~1½ – 2 minutes)

Step 1 – Overview of the Page

“This is the Order History page, which displays all orders in the system. It automatically separates *current* and *past* orders for clarity.”

- Briefly toggle between *Current* and *Past* tabs.
- Point out the count badges (e.g., “5 Current, 8 Past”).

Step 2 – Searching and Filtering

“I can search by order number, customer, or status, and it filters instantly.”

- Type a partial customer name or status keyword to show live filtering.
- Then use the date range pickers:

“Here I can filter orders placed within a specific time window.”

- On the *Past* tab, open the *Status* dropdown and pick *Settled*:

“This shows only settled orders.”

- Clear filters once to demonstrate the *Clear Filters* button.

Step 3 – Sorting and Pagination

“Every column header is sortable. If I click on Date, it re-orders ascending or descending, with an arrow indicator.”

- Click once or twice to show arrow toggling.
- Then mention pagination:

“Down here I can change how many results display at once—20, 50, or 100—for large data sets.”

Step 4 – Order Detail View

“Clicking an order opens its details right here under the filters.”

- Click a row → show inline panel:
 - Order summary (ID, customer, total, status badge)
 - Authorization info
 - Settlement list with amounts/dates/badges

“All of this data comes from the same API endpoint that the backend uses, so it’s always up to date.”

Step 5 – CSV Export

“Finally, we can export the current view to a CSV for reporting.”

- Click **Export CSV**, show download prompt.

Summary of Demonstrated Features

Category	Feature	Demo Step
Required	Order Retrieval	Step 1
Required	Sorting	Step 3
Required	Filtering Tabs	Step 1
Required	Search	Step 2
Required	Order Details Panel	Step 4
Required	CSV Export	Step 5

Bonus	Date Range Filter	Step 2
Bonus	Status Dropdown	Step 2
Bonus	Clear Filters Button	Step 2
Bonus	Sort Icons	Step 3
Bonus	Pagination	Step 3
Bonus	Persistent Counts	Step 1
Bonus	Inline Panel Placement	Step 4

Database Functions & Setup: Felix

Format: PowerPoint slide(s) with a simple schema or flow diagram.

Schema and Relationships (60–70 seconds)

Slide Content Suggestion:

A single diagram with three tables and arrows connecting them:

- **ORDERS**
 - order_id
 - customer_name
 - card_last4
 - amount
 - status
 - created_at
- **AUTHORIZATIONS**
 - id
 - order_id → links to ORDERS
 - provider_token
 - amount
 - outcome
- **SETTLEMENTS**
 - id
 - order_id → links to ORDERS
 - amount
 - outcome
 - created_at

Talking Points:

“The database centers on three main tables:

- **Orders** – one row per customer order, storing basic information such as the total amount, last 4 card digits, and current status like PENDING, AUTHORIZED, SETTLED, or ERROR.
- **Authorizations** – records the payment authorization outcome from the provider and links to an order by ID.
- **Settlements** – records each warehouse settlement transaction against an authorized order.”

“Each authorization and settlement is connected back to its order by order_id.

This structure guarantees data integrity and ensures that all three UIs pull from a consistent dataset.”

Data Flow Mapping (40–45 seconds)

Slide Content Suggestion:

A simple left-to-right flow diagram:

Payment Page → Orders & Authorizations → Warehouse Settlement → Settlements → Order History

Talking Points:

“All three UIs communicate with the same backend and database.

- The **Payment Page** creates an order and an authorization record when a card is approved.
- The **Warehouse Settlement Page** adds new settlement records and updates the remaining amount on that order.
- The **Order History Page** retrieves this data, joins it, and displays the complete picture for users.”

Constraints and Validation (20–30 seconds)

Slide Content Suggestion:

Three bullet points:

- Order statuses limited to: *PENDING, AUTHORIZED, SETTLED, ERROR*
- Authorization outcomes: *SUCCESS, INSUFFICIENT_FUNDS, INCORRECT_DETAILS, PROVIDER_ERROR*
- Settlements limited to total authorized amount

Talking Points:

“The schema also enforces a few rules to keep the data consistent:

- Orders only use specific status values.
- Authorization and settlement outcomes use standardized result codes.
- The backend logic ensures that the total of all settlements never exceeds the authorized amount for an order.”

Testing and Sample Data (Optional, 10–15 seconds)

“To verify that everything worked, the database was populated with test orders representing different outcomes — successful payments, insufficient funds, invalid card details, and multi-settlement orders — so that all possible workflows could be tested end-to-end.”

Conclusion:

Future Development:

1. Role-Based Login and Permission Levels

“A natural next step would be to add a login system with role-based permissions. Warehouse users would have basic access to recent authorized orders, while administrators could access the entire dataset for reporting. Permission levels could also control how much data each role can load, improving performance and conserving resources.”

2. Server-Side Pagination and Filtering

“Currently, the UIs handle most filtering and pagination on the client side, which works fine for a small demo dataset.

For larger datasets, this logic could move to the backend so that only a slice of data is sent to the browser at a time, improving scalability and speed.”

3. Migration from SQLite to a Production Database

“Our project uses SQLite for simplicity and local testing.

A production environment could migrate to PostgreSQL or MySQL to handle larger volumes of data and concurrent access, while keeping the same schema and API structure.”

4. Integration with a Real Payment Provider

“Right now, the system uses a mock payment provider to simulate authorizations and settlements.

The architecture is flexible enough that a real provider API could be integrated later, with minimal changes to the database or UI.”

5. Monitoring and Audit Trails

“A future production system could also include audit tracking and activity logs — recording who authorized or settled each transaction.

This would enhance security, accountability, and data traceability, especially if role-based logins were added.”

AI Disclaimer:

AI Use and Acknowledgment

Slide Title:

Use of AI in Development

Slide Text (to display on screen):

- AI tools were used throughout this project to **enhance workflow efficiency** and **improve code quality**.
- Assisted in **debugging, error tracing, and code optimization** during development.
- Supported **documentation, presentation planning, and feature brainstorming**.
- Helped the team **audit and refine existing functionality** without changing core requirements.
- All **content, design decisions, and implementation** were created and verified by the group members.
- AI served as a **collaborative assistant**, not a replacement for student work.

Presenter Note:

“Throughout this project, we made use of AI tools to help speed up development and improve accuracy.

We used AI for tasks like debugging code, troubleshooting backend errors, refining our frontend logic, and brainstorming polish features.

It also helped us document our progress, organize our milestones, and prepare for this presentation.

All of the ideas, designs, and implementations you see here were decided on and built by our group — AI simply acted as a smart assistant to enhance our workflow and catch issues faster.”

Thank the Audience